SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Project 2: Full-Fledged Content Management System (CMS)

Enterprise Multi-Role Web Publishing Platform & Content Administration Architecture

PROJECT TYPE	DATABASE ENGINE	BACKEND FRAMEWORK	CLIENT FRONTEND
Full-Stack Web App	**MySQL 8.x (InnoDB, PDO)**	**PHP 8.x Native / OOP**	**HTML5, CSS3, JavaScript**

1. INTRODUCTION

1 .1 Purpose

The Content Management System (CMS) is engineered as an enterprise-grade platform allowing multi-disciplinary teams to create, edit, organize, review, schedule, and publish rich digital content without manual source-code intervention. It centralizes asset management, editorial moderation, and taxonomy across diverse publications.

1 .2 Project Scope

Suitable for institutional portals, news and magazine publications, high-volume educational resources, corporate documentation bases, and multi-author platforms requiring strict access control, editorial oversight, and structured workflows.

1 .3 Core Project Objectives

- Provide secure user authentication, session controls, and granular role permissions.
- Deliver an intuitive rich-text article editor with live previews and autosave capability.
- Enforce multi-stage editorial governance (draft, pending review, scheduled, published).
- Enable robust digital asset management, structured categorization, and custom navigation.
- Incorporate search engine optimization (SEO), metadata configuration, and XML sitemaps.
- Safeguard integrity via PDO prepared queries, CSRF defenses, and comprehensive audit logs.

2. OVERALL SYSTEM DESCRIPTION

Role	Operational Scope & Responsibilities
Super Administrator	Complete system governance, global settings, role configurations, and audit trail.
Administrator	User lifecycle management, system settings, category structuring, and moderation.
Editor	Review, revision feedback, publication scheduling, and unpublishing of submissions.
Author	Full ownership of self-authored posts, drafts, media uploads, and submission for review.
Contributor	Content drafting and submission for editorial review without direct publish rights.
Subscriber / Guest	Profile management, article bookmarks; guests browse published public posts.

Note: Standard static hosts cannot execute PHP. A dedicated runtime supporting PHP 8.x execution and MySQL connectivity is required.

Layer	Technology
Client Frontend	HTML5, CSS3, ES6+ JavaScript
UI Framework	Bootstrap 5.x (Responsive)
Application Core	PHP 8.x (Strict Typing, OOP)
Database Access	PDO with Prepared Statements
Relational DB	MySQL 8.x (InnoDB Engine)

Web Server	Apache 2.4 / Nginx
Dev & Testing	XAMPP / PHPUnit / Git

2.1 User Roles & Permissions Matrix

2.3 Technology Stack

3. FUNCTIONAL REQUIREMENTS (PART I: CORE OPERATIONS)

3.1 Authentication & Account Management

FR-01 Users shall register an account providing name, verified email address, and secure password.
FR-02 System shall authenticate credentials securely and provide explicit login/logout session handling.
FR-03 Passwords shall be cryptographically hashed using PHP `password_hash()` with `PASSWORD_DEFAULT`.
FR-04 Secure password reset workflows via time-limited, single-use cryptographic tokens.
FR-05 Support optional email verification prior to granting full authoring privileges.
FR-06 Users shall update their profile, bio, display name, avatar, and security credentials.
FR-07 Administrators shall activate, suspend, deactivate, or assign roles to user accounts.
FR-08 Server-side enforcement of granular role-based access control (RBAC) on all protected routes.

3.2 Content Creation & Management

FR-11 Authors shall compose new rich articles, drafts, and multi-format publications.
FR-12 Authors shall edit, revise, and update their own articles consistent with their permission matrix.
FR-14 Content body shall support headings, lists, blockquotes, inline imagery, code blocks, and embedded media.
FR-15 Integrated rich-text editor with automated client/server HTML sanitization against XSS injections.
FR-16 Automatic draft saving and preview generation prior to formal submission or live publishing.
FR-19 Automated scheduled publication mechanism based on designated future timestamps.
FR-21 Every post shall generate and validate a unique, URL-safe slug string.
FR-23 Distinct status tracking: `Draft`, `Pending Review`, `Scheduled`, `Published`, `Archived`, and `Rej ected`.
FR-27 Soft-deletion support: deleted items move to a recoverable Trash bin subject to retention lifecycles.

3.3 Static Page Management

FR-28 Create/maintain static pages (e.g., About, Contact, Privacy).
FR-29 Configurable layout templates (full-width, sidebar, boxed).
FR-30 Support hierarchical parent-child relationships in pages.
FR-33 Selectable static page as default public homepage.

3.4 Taxonomies: Categories & Tags

FR-35 Manage hierarchical categories with parent-child nesting.
FR-37 Assign posts to single or multiple thematic categories.
FR-38 Assign reusable flat keyword tags to individual posts.
FR-40 Public archive views filtered by categories and tags.

3.5 Digital Media Library

FR-41 Centralized upload of images (JPG, PNG, WebP) and docs.
FR-44 Persist filename, MIME type, file byte size, and alt tags.
FR-46 Strict server MIME validation and file size restrictions.
FR-48 Pre-deletion dependency audit for active post usage.

3.6 Comments & Community Moderation

FR-50 Public reader commenting on enabled articles.
FR-52 Multi-state moderation: `Pending`, `Approved`, `Spam`.
FR-54 Nested multi-level threaded discussions and replies.
FR-56 Global or post-level toggles to disable discussion.

3. FUNCTIONAL REQUIREMENTS (PART II: EDITORIAL & GOVERNANCE)

3.7 Editorial Review Workflow

FR-57 Contributors submit content directly into the review queue.

FR-58 Editors approve, decline, or return drafts with revision feedback.

FR-59 System logs reviewing editor ID and exact review timestamps.

FR-62 Internal editorial notes hidden from public view.

3.8 Revision History & Recovery

FR-64 System logs point-in-time snapshots upon saving content.

FR-65 Visual comparison interface between historical revisions.

FR-66 One-click rollback restores prior content states safely.

FR-67 Every snapshot records editor identity and timestamps.

3.9 Search & Navigation Engine

FR-69 Full-text keyword matching across titles, bodies, and tags.

FR-71 Server-side pagination on all search and listing views.

FR-73 Drag-and-drop navigation menu builder for primary headers.

FR-75 Breadcrumb generation reflecting taxonomy hierarchy.

3.1 0 SEO & Publishing Controls

FR-77 Custom meta titles, descriptions, and canonical URLs.

FR-79 Automated XML sitemap generation (`sitemap. xml`).

FR-80 Custom `robots. txt` and noindex directives.

FR-82 Automatic OpenGraph social sharing meta tags.

3.1 1 Dashboard & Content Analytics

FR-85 Executive KPI metrics: post totals, views, users, pending reviews.

FR-86 Author dashboard isolating self-authored metrics and feedback.

FR-88 Lightweight privacy-compliant page hit tracking per post.

FR-90 CSV export of content summaries and moderation logs.

3.1 2 Notifications Architecture

FR-92 In-app alerts for editorial assignments, comments, and approvals.

FR-94 Transactional emails for password resets and critical changes.

FR-95 Unread/read status tracking with batch clearing.

FR-96 User preferences for optional email notifications.

3.1 3 User Administration & Immutable Audit Trail

FR-97 Searchable, sortable user registry with instant account locking and role switching.

FR-1 00 Tamper-resistant audit log capturing user ID, IP address, timestamp, event action, and entity ID.

FR-1 02 Audit entries cannot be edited or deleted via standard administrative UI screens.

3.1 4 Advanced & Extensible Roadmap Features

Designed for post-core implementation to enhance enterprise scalability:

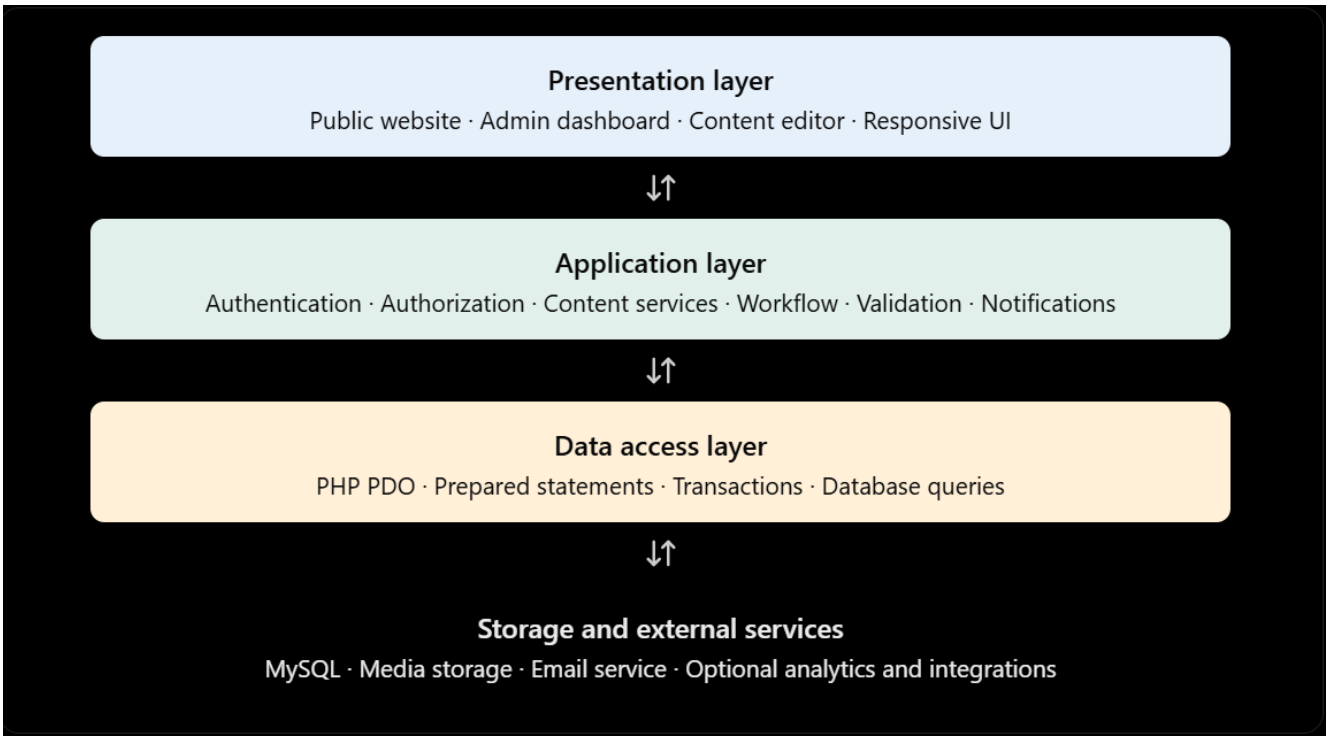
- Multi-language localization and translation linking
- Webhook dispatcher on publishing events
- Broken link detector and automated 301 redirection table
- Headless REST API endpoints for external clients
- Scheduled automated database snapshot and backup utility
- Custom content blocks and reusable component widgets

4. NON-FUNCTIONAL & SECURITY REQUIREMENTS

ID	Category	Technical Specification & Standard
NFR-01	Performance	Page rendering under 1 .2s under standard server load; optimized indexes on foreign keys and slugs.
NFR-02	Security & Auth	All privileged endpoints mandate strict session validation, CSRF checks, and role verification.
NFR-03	Usability	Administrative dashboard compliant with intuitive UI standards; non-technical users draft effortlessly.
NFR-04	Responsiveness	Fully responsive across mobile (375px+), tablet, and desktop viewports via Bootstrap grids.
NFR-05	Reliability	Database transactions (ACID) for multi-table updates; graceful error boundaries without data corruption.
NFR-06	Data Integrity	Strict foreign key constraints with `ON DELETE RESTRICT` on essential historical entities.
NFR-07	Maintainability	Clean Model-View-Controller (MVC) separation of concerns; modular PHP services and repository pattern.

5. SYSTEM ARCHITECTURE

The application should use a layered architecture so that the user interface, application logic, and database do not become one giant collection of tangled PHP files.



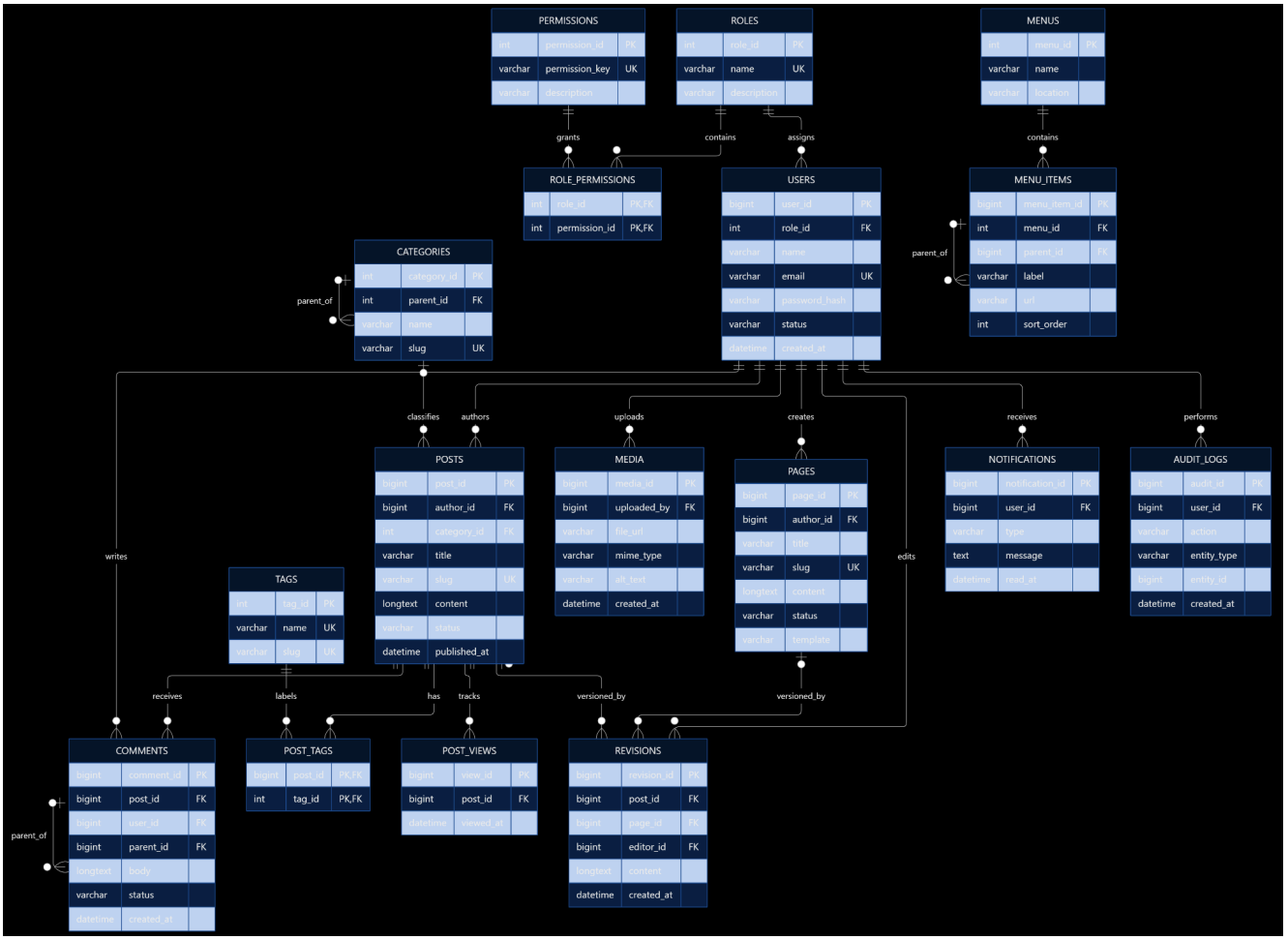
6. DATABASE DESIGN & SCHEMA SPECIFICATIONS

Table Name	Primary Keys & Foreign Keys	Functional Storage Purpose
`users`	`user_id` (PK), `role_id` (FK)	User identity, credentials hash, profile, and account status.
`roles / permissions`	`role_id` (PK), `permission_id` (PK)	RBAC definition and many-to-many permission assignments.
`posts`	`post_id` (PK), `author_id` (FK), `cat_id` (FK)	Articles, unique slugs, content body, status, publish timestamp.
`pages`	`page_id` (PK), `author_id` (FK)	Static hierarchical content, custom slug, layout template.
`categories / tags`	`category_id` (PK), `tag_id` (PK)	Hierarchical categories (self-FK) and searchable tagging index.
`media`	`media_id` (PK), `uploaded_by` (FK)	Asset path, MIME type, file size, alt text, and upload date.
`comments`	`comment_id` (PK), `post_id` (FK), `parent_id` (FK)	Reader commentary, moderation status, and threaded replies.
`revisions`	`revision_id` (PK), `editor_id` (FK)	Unified historical snapshots linked to either a post or a page.
`audit_logs`	`audit_id` (PK), `user_id` (FK)	Immutable log of security, authentication, and moderation events.

6.2 Relational Integrity Rules: Slugs enforce unique indexes across active routes. Unified revision records require either `post_id` OR `page_id` to be non-null. User deletion cascades to profile metadata but retains post authorship via soft assignment or archival attribution to preserve historical publications.

7. ENTITY RELATIONSHIP DIAGRAM (ERD)

The following ER diagram shows the core entities and relationships for the CMS. It is written in Mermaid, so you can paste it into a Mermaid-compatible editor or the Mermaid Live Editor to render and export the diagram.


PERMISSIONS
int permission_id PK
varchar permission_key UK
varchar description
ROLES
int role_id PK
varchar name UK
varchar description
MENUS
int menu_id PK
varchar name
varchar location
grants
contains
assigns
contains
ROLE_PERMISSIONS
int role_id PK,FK
int permission_id PK,FK
USERS
bigint user_id PK
int role_id FK
varchar name
varchar email UK
varchar password_hash
varchar status
datetime created_at
MENU_ITEMS
bigint menu_item_id PK
int menu_id FK
bigint parent_id FK
varchar label
varchar url
int sort_order
parent_of
CATEGORIES
int category_id PK
int parent_id FK
varchar name
varchar slug UK
parent_of
classifies
authors
uploads
creates
receives
performs
writes
edits
POSTS
bigint post_id PK
bigint author_id FK
int category_id FK
varchar title
varchar slug UK
longtext content
varchar status
datetime published_at
MEDIA
bigint media_id PK
bigint uploaded_by FK
varchar file_url
varchar mime_type
varchar alt_text
datetime created_at
PAGES
bigint page_id PK
bigint author_id FK
varchar title
varchar slug UK
longtext content
varchar status
varchar template
NOTIFICATIONS
bigint notification_id PK
bigint user_id FK
varchar type
text message
datetime read_at
AUDIT_LOGS
bigint audit_id PK
bigint user_id FK
varchar action
varchar entity_type
bigint entity_id
datetime created_at
TAGS
int tag_id PK
varchar name UK
varchar slug UK
receives
labels
has
tracks
versioned_by
versioned_by
COMMENTS
bigint comment_id PK
bigint post_id FK
bigint user_id FK
bigint parent_id FK
longtext body
varchar status
datetime created_at
parent_of
POST_TAGS
bigint post_id PK,FK
int tag_id PK,FK
POST_VIEWS
bigint view_id PK
bigint post_id FK
datetime viewed_at
REVISIONS
bigint revision_id PK
bigint post_id FK
bigint page_id FK
bigint editor_id FK
longtext content
datetime created_at

8. USE CASE SPECIFICATIONS

UC-01 : Create & Publish Article

Primary Actor: Author / Contributor

- Author navigates to editor and composes title and body.
- Author specifies category, tags, and SEO metadata.
- System verifies slug uniqueness and sanitizes payload.
- Author submits post; contributors transition state to *Pending Review*.
- Editor reviews, approves, and assigns *Published* status.

Postcondition: Article rendered live on public listing and sitemap.

UC-02: Version Comparison & Restoration

Primary Actor: Editor / Administrator

- Editor opens revision history panel for specific post.
- System retrieves chronologically ordered revision hashes.
- Editor selects target historical version and confirms rollback.
- System creates a new sequential revision snapshot storing restored content without destroying edit history.

Postcondition: Prior version safely restored with full audit preservation.

1 0. ENTERPRISE BUSINESS RULES

- **BR-01 :** Administrative dashboard strictly restricted to authenticated sessions with verified management permissions.
- **BR-02:** Contributors cannot self-publish; all submissions mandate explicit Editor or Administrator approval.
- **BR-03:** Public router must strictly reject requests for posts in `Draft`, `Pending`, or `Archived` state.
- **BR-04:** Every URL slug must be strictly unique within its namespace (posts vs. pages); auto-append numerals on conflict.
- **BR-05:** User deletions must not orphan or delete published public articles; retain historical authorship links.
- **BR-06:** Scheduled publications trigger strictly according to configured system timezone offsets.

1 1 . API & INTEGRATION SPECIFICATIONS

Method	Endpoint	Access Level	Functional Scope
GET	`/api/v1 /posts`	Public	Fetch paginated published articles with filter parameters.
GET	`/api/v1 /posts/{id}`	Public	Retrieve single published article with taxonomy details.
POST	`/api/v1 /posts`	Authenticated (Author+)	Create new post or draft; validates JSON schema.
PATCH	`/api/v1 /posts/{id}`	Editor / Post Author	Update content, categories, tags, or publishing state.
DELETE	`/api/v1 /posts/{id}`	Editor / Admin	Soft delete article into trash retention bin.
GET	`/api/v1 /search?q={query}`	Public	Full-text search across titles, contents, and tags.

External integrations supported: SMTP transactional mailers, Cloudinary/S3 asset storage, OpenGraph metadata scrapers, and analytics webhooks.

1 2. VERIFICATION & TESTING SPECIFICATIONS

Test Discipline	Validation Focus	Expected Pass Criterion
Unit Testing	Slug generation, validation rules, hashing.	1 00% pass on core utility functions via PHPUnit.
Security Testing	SQL injection, XSS vectors, CSRF validation.	Zero arbitrary script execution; prepared statements enforced.
Authorization Testing	Access control matrix and role escalation.	Contributors strictly forbidden from direct publishing endpoints.
Database Integrity	Foreign key triggers, transaction rollbacks.	No orphaned records on post deletion or failed multi-table writes.

1 2.1 Acceptance Criteria: Complete functional verification achieved when registered authors compose, preview, and submit articles; editors review, schedule, and publish without errors; search returns matching results; and unauthenticated users cannot access administrative endpoints.

1 3. TECHNICAL CONSTRAINTS & ASSUMPTIONS

- PHP 8.x and MySQL 8.x must be available in the hosting environment with PDO MySQL extensions enabled.
- Production servers require active HTTPS certificates for secure cookie session transmissions.
- Transactional email features (password recovery, review notifications) depend on a configured SMTP service.

1 4. SUGGESTED DEVELOPMENT ROADMAP

01
Foundation
DB schema, PDO connection, auth, RBAC, session security.

02
Core CMS
Posts, pages, categories, tags, rich editor, drafts.

03
Editorial
Review queue, revisions, media library, comments.

04
Governance
SEO, sitemaps, analytics, audit log, search.

05
Production
Unit tests, security audit, backups, and live deploy.

1 5. EXPECTED PROJECT OUTCOME

The finished platform delivers a robust, multi-user, database-driven Content Management System built strictly on PHP and MySQL standards. It guarantees seamless lifecycle governance from initial drafting to editorial review, publication, search indexing, and administrative auditing.

Specification Status: Approved for Implementation

Architecture Target: PHP 8.x · MySQL 8.x · Responsive MVC